Summary of “DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving”

Nathan Yap (nyap), Hannah Shu (hyshu), Patrick Halim (pnhalim)

Problem and Motivation

LLM inference latency is two-fold; it consists of TTFT (time to first token) during the prefill phase and TPOT (time per output token) during the decode phase. Different applications have various latency requirements—for example, a real-time chat application vs. a real-time coding assistant would have different TTFT and TPOT requirements. These various requirements pose an issue when trying to use the fewest number of GPUs to serve as many requests as possible.

Existing systems colocate prefill and decode stages on one GPU and batch the phases across requests, maximizing the overall system throughput. However, running both phases on one GPU leads to prefill-decode interference due to the prefill steps taking much longer, subsequently causing decode steps to be delayed, which increases TPOT. Similarly, the inclusion of decode steps causes an increase in TTFT. Additionally, the colocation forces prefill and decode stages to utilize the same forms of parallelism, but since prefill is compute-bound and decode is memory-bound, this leads to overprovision of resources. DistServe aims to address these issues by disaggregating prefill and decode stages.

Related Works

- Past:
    - **Orca**: Prioritize prefill TTFT with continuous batching (finer-grained scheduling), but still colocates prefill and decode
    - **vLLM**: Prioritize TPOT decode with paged-attention for KV cache management, but still colocates prefill and decode
- Future:
    - **Sarathi-Serve** (below)

Solution Overview

The authors propose disaggregating the prefill and decode phases on separate GPUs. This eliminates the prefill-decoding interference and naturally divides the SLO satisfaction problem into two optimization problems, where the prefill instance optimizes for TTFT, and the decode instance optimizes for TPOT. This allows each phase to have different parallelism and resource allocations for their specific latency requirements.

However, these benefits are not free: there is communication overhead due to the need to transfer KV-cache from the prefill instance to the decode instance. Additionally, the goal of optimizing for per-GPU goodput is still difficult since it depends on so many factors, such as the workload characteristics, SLO requirements, parallelism stages, resource allocation, and network bandwidth.

DistServe has key features designed to address these challenges: given the LLM, workload pattern, and SLO requirements, DistServe will decide the placement to serve this specific LLM. The placement includes the parallelism strategy for the prefill/decoding instances, the number of each instance to deploy, and how to place them on the physical cluster.

If the nodes in the cluster have a high-bandwidth network like Infiniband, the KV-cache transfer time is negligible, even if it happens between two nodes. This means that prefill and decode instances can be optimized separately. To do this, the algorithm will use a simulation to measure the goodput for various parallelism configurations and choose the optimal configuration for each phase.

For clusters with lower-bandwidth cross-node connections, the authors observe that the KV-cache transmission only happens between the same layer of prefill/decoding instances. So the algorithm sketch will be the same, but with the added constraint that the same stage of prefill/decoding instances be on the same node, so that the transmission can use high-bandwidth intra-node communications.

Finally, online scheduling optimizations are used to adapt to real-world workloads. Scheduling balances execution time across all batches in the pipeline in order to reduce pipeline bubbles caused by non-uniform workloads. It also addresses bursty workloads that would cause a flood of KV-cache transfers, DistServe uses a “pull” rather than a “push” method by using the memory of the GPU instances as a queueing buffer. Finally, since this method depends heavily on the workload characteristics, DistServe employs periodic replanning that monitors the workload pattern and can rerun the placement algorithm on the fly.

They evaluated DistServe against vLLM and DeepSpeed-MII for per-GPU goodput on different model sizes for different applications (chatbot, code-completion, and summarization) and found that for all the examined application types, DistServe has higher SLO attainment per-GPU.

They also found that the KV-cache transmission accounts for < 0.1% of the total latency.

Limitations

DistServe is not feasible on smaller systems, i.e. it can’t be used on individual GPUs due to the fact that the entire premise is disaggregating prefill and decode stages onto different GPUs. If

there's only a small amount of resources, then the overhead of DistServe (replicating model weights, scheduling, etc.) may be impractical compared to non-disaggregated systems. Additionally, the paper does not provide any methods for addressing fault tolerance between GPUs. Since different GPUs handle separate stages, failure in one (for example, one of the decoding GPUs) could stall other prefill instances. It also does not deal well with long context workloads that will greatly increase the prefill phase workload and increase the size of KV-cache and may lead to bandwidth bottlenecks when doing the KV transmissions.

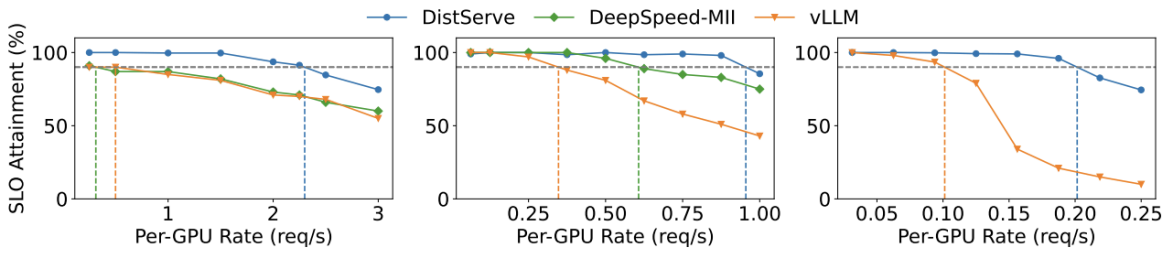
Future Research Directions

The current placement algorithm is offline and based on simulation results using a fixed traffic rate. Running the placement algorithm online would allow resources to be dynamically reallocated to prefill/decode based on the live workload characteristics. DistServe also currently doesn't have any way of addressing failures, which would be an important area of future work to ensure fault tolerance so jobs can be redistributed if a node fails. Finally, to address the limitation with long-context windows, it would be interesting to explore hierarchical or partial disaggregation instead of full disaggregation.

Summary of Class Discussion

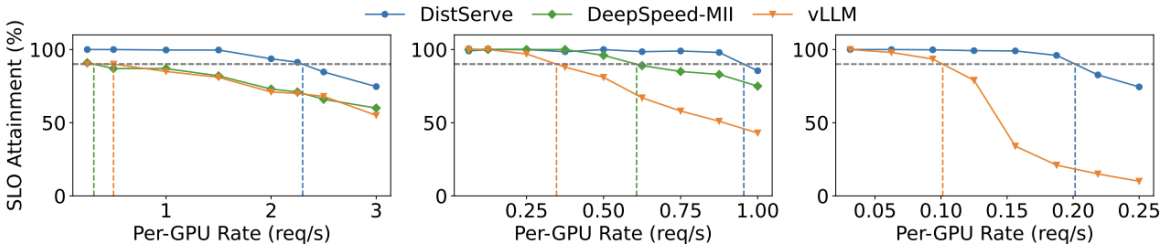
Q: In the algorithm for high-node affinity, you don't want the ratio of prefill/decode to be 1:1, otherwise the resources will be underutilized. It isn't clear how they figure out the ideal ratio.
A: Given a specific parallelism configuration, the algorithm for high-node affinity identifies the best configuration for per-phase goodput. Then, it takes the estimated traffic rate R and divides by the goodput for each stage to estimate the number of GPUs needed for prefill/decode.

Q: Is the metric shown in these plots useful? Why not look at system wide input rather than per GPU?



A: Tradeoff raw throughput for increasing SLO attainment. It is weird that the metric is divided by GPUs, it might be better to have everyone get the same GPU configuration and measure the total system throughput rate.

Q: Why is the SLO attainment for vLLM so low on the right graph for higher request rates?



A: The right graph is using a model with 175 billion parameters while the first and second graphs are using 13B and 66B respectively. The larger model naturally takes longer to process requests, which the graph reflects.

Q: Is there any specialized hardware for prefill or decode operations?
A: Yes, but the more specialized hardware is, the harder it is to fully utilize it because the workload needs to match the hardware configuration. If the workload doesn't fit that configuration, it can lead to loss of efficiency.

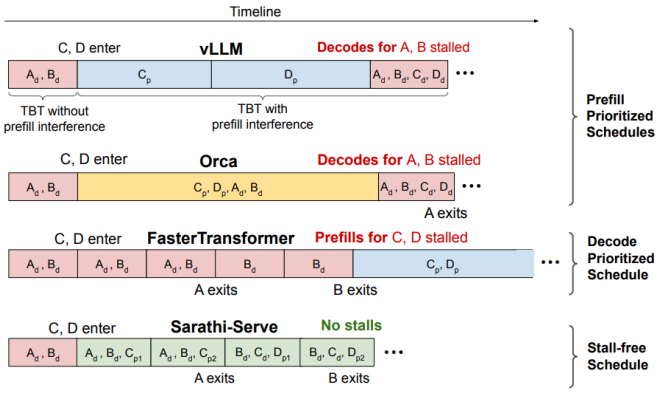
Q: Can you combine the two papers, DistServe and Sarathi-Serve?
A: Intuitively this does not make much sense to combine them, since SS uses hybrid batching, while DistServe splits up the phases across GPU. In practice, it is possible to combine them by pipelining the chunk prefill then doing KV transfer.

Q: Are decode tokens batched in parallel? Are there bubbles at the start when there are no decode steps in the queue?
A: The decoding step is done in continuous batching. When new decodes come in, those can get added to existing batches that are running. Yes, there are bubbles at the start when there are no decode steps in the queue.

Q: Do these results for DistServe apply to any scale of traffic or is some minimum amount of traffic needed to see these results?
A: It's reasonable to assume there needs to be a consistent workload; there needs to be some justification to be using at least two nodes, otherwise it wouldn't make sense to use DistServe. If the workload is too small, then only one prefill/decode phase is active at a time, but if it's too high, then it will be KV-cache bound, but KV-cache is reduced for DistServe. Thus, there is a balance to strike for optimal results.

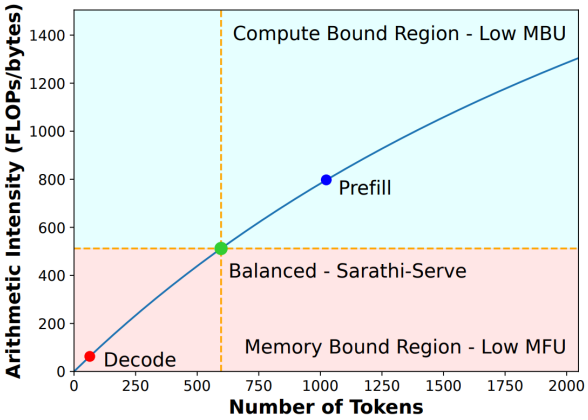
Summary of “Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve”

Problem and Motivation



LLM inference requires a compute-bound prefill and a memory-bound decode stage. Existing implementations either prioritize the prefill stage or decode stage when scheduling batches. Prioritizing the prefill stage can lead to *generation stalls* as prefill can take an arbitrarily long amount of time, however prioritizing prefills leads to higher batch-size (and thereby higher throughput in the decode stage). Prioritizing decoding (only scheduling new prefills after all decoding finishes) optimizes the TPT (time per token) during generation, however can reduce throughput as some requests finish early, leading to a smaller batch size and lower utilization.

Sarathi-serve seeks to balance the tradeoff between throughput and request latency with *chunked-prefills* and *stall-free scheduling*.



Related Works

- **Decode-prioritizing works:** pick a batch of requests and execute it until all requests in the batch complete
    - E.x. FasterTransformer, Triton Inference Server
- **Prefill-prioritizing works:** eagerly admit new requests in a running batch at the first available opportunity (whenever GPU memory becomes available)
    - E.x. Orca, vLLM

Solution Overview

Chunked prefills involve computing large prefills across iterations in smaller chunks. They observe that in many practical scenarios the prefill size exceeds the length required to completely saturate a GPU, therefore smaller chunks can still fully utilize compute potential. **Stall-free batching** coalesces chunked prefills and decodes to improve throughput and minimize latency. This allows Sarathi-serve to execute prefills without delaying the execution of decodes.

Sarathi-serve calculates the maximum number of tokens that can be executed based on user service level objectives (SLO). This token budget is based on TBT SLO and chunked-prefill overhead, this budget can therefore vary due to device configuration and hardware properties.

Sarathi-serve will schedule token decodings alongside ongoing prefill stages and new prefill requests to best balance latency and throughput during inference.

When evaluating the performance of Sarathi-serve (SS), they found that SS was able to maintain SLO at higher capacities (queries per second) compared to Orca and vLLM.

Using ablation studies, they were able to quantify the overhead of implementing chunked prefills under different conditions, showing that for larger token budgets, the overhead of chunked prefills became negligible.

Limitations

The overhead of scheduling chunked prefills adds additional engineering effort and implementation complexity. The benefits of SS also vary depending on the workload, as it works best when workloads are balanced.

Additional profiling is also required to determine the token budget for a runtime.

Future Research Directions

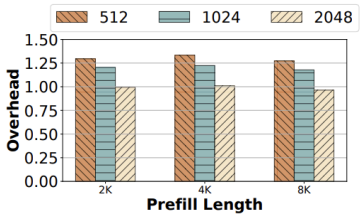
Dynamic token budgeting (based on workload characteristics) was mentioned as a potential way to apply SS to various workloads dynamically during runtime. However this requires more overhead and is harder to reason about on the fly.

Broader hardware/network evaluation (such as testing on heterogeneous clusters + other GPU types) would also help to fully evaluate the effectiveness of Sarathi-serve.

Summary of Class Discussion

Q: Does each chunk (using KV cache between chunks) have to execute on the same host?
A: It is reasonable to assume they put it in the same node to keep low latency, though not much is mentioned in the paper. The first chunk will be accessed n-1 times, the second chunk accessed n-2 times, etc. so they want to keep the chunks on the same GPU.

Q: Why does 8k prefill length have overhead of less than 1 for chunk size 2048?



A: No clear reason why, need to go back to the paper and see if the experiments were run multiple times and had low overhead aggregated over those runs.

Q: If there's negligible overhead, why not use Sarathi-Serve every time; why use vLLM?
A: This depends on the workload, as there are tradeoff latency considerations in real-time use-cases. If you just care about system throughput, then you use vLLM. The authors didn't test where the tradeoff is on either extreme (prefill-heavy workload vs. decode heavy-workload), therefore it is unclear how this system performs and where generation stalls will happen.

Q: Does it make sense to use nested (combined) methods in combination with SS, e.g. when doing RLHF?
A: This again is heavily workload dependent. For RL, the system should prioritize optimizing throughput like using vLLM. SS is good for latency-sensitive tasks and could be used with other methods, but you would have to explore the tradeoffs with that.